

◆ 1. What is ADO.NET (Overview)

ADO.NET (ActiveX Data Objects .NET) is a **data access framework** in .NET used to interact with databases such as SQL Server, Oracle, MySQL, etc.

✓ Key Goals

- Provide **efficient data access**
 - Support both **connected and disconnected models**
 - Enable **data manipulation (CRUD)**
 - Work with **XML-based data representation**
-

◆ ADO.NET Architecture (High-Level)

ADO.NET has **two major components**:

1. **Data Providers (Connected Layer)**
 2. **DataSet (Disconnected Layer)**
-

◆ 2. ADO.NET Data Providers

A **Data Provider** is a set of classes that:

- Connect to a database
 - Execute commands
 - Retrieve results
-

◆ Common Data Providers

Provider	Description
----------	-------------

SQL Server	Microsoft.Data.SqlClient
------------	--------------------------

OLE DB	System.Data.OleDb
--------	-------------------

Provider	Description
ODBC	System.Data.Odbc

◆ Core Components of a Data Provider

Every provider includes these 4 main objects:

1. Connection

- Establishes connection with DB

2. Command

- Executes SQL queries

3. DataReader

- Reads data (forward-only, read-only)

4. DataAdapter

- Bridges DataSet and database
-

◆ Flow (Connected Model)

Connection → Command → DataReader

◆ 3. Microsoft.Data.SqlClient Classes

This is the **modern SQL Server provider** used in .NET Core / .NET 8.

◆ Important Classes

1. SqlConnection

Used to open/close DB connection

using Microsoft.Data.SqlClient;

```
SqlConnection conn = new SqlConnection("connection_string");  
conn.Open();
```

2. SqlCommand

Used to execute SQL queries

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Employees", conn);
```

Types of commands:

- ExecuteReader() → returns DataReader
 - ExecuteNonQuery() → INSERT/UPDATE/DELETE
 - ExecuteScalar() → single value
-

3. SqlDataReader

- Fast, forward-only, read-only
- Requires open connection

```
SqlDataReader reader = cmd.ExecuteReader();
```

```
while(reader.Read())  
{  
    Console.WriteLine(reader["Name"]);  
}
```

4. SqlDataAdapter

- Used for **disconnected architecture**
- Fills DataSet

```
SqlDataAdapter adapter = new SqlDataAdapter(cmd);
```

```
DataSet ds = new DataSet();
```

```
adapter.Fill(ds);
```

5. SqlParameter

- Used for parameterized queries (avoids SQL Injection)

```
cmd.Parameters.AddWithValue("@Id", 1);
```

◆ 4. DataSet (Disconnected Memory Model)

A **DataSet** is an **in-memory representation of data**.

It does NOT require an active DB connection.

◆ Components of DataSet

1. DataTable

- Represents a table

2. DataRow

- Represents a row

3. DataColumn

- Represents a column

4. DataRelation

- Defines relationships between tables
-

◆ Example

```
DataSet ds = new DataSet();
```

```
SqlDataAdapter adapter = new SqlDataAdapter(cmd);
```

```
adapter.Fill(ds);
```

```
DataTable table = ds.Tables[0];
```

```
foreach(DataRow row in table.Rows)
{
    Console.WriteLine(row["Name"]);
}
```

◆ Key Features of DataSet

- Works **offline (Disconnected)**
 - Stores multiple tables
 - Supports relationships
 - Can be serialized to XML
-

◆ 5. Connected Architecture

◆ Definition

In this model, the application:

- Keeps the connection **open**
 - Interacts directly with DB
-

◆ Flow

Open Connection → Execute Command → Read Data → Close Connection

◆ Example

```
SqlConnection conn = new SqlConnection("connection_string");
conn.Open();
```

```
SqlCommand cmd = new SqlCommand("SELECT * FROM Employees", conn);  
SqlDataReader reader = cmd.ExecuteReader();
```

```
while(reader.Read())  
{  
    Console.WriteLine(reader["Name"]);  
}  
  
conn.Close();
```

◆ Advantages

- High performance
- Real-time data
- Less memory usage

◆ Disadvantages

- Connection remains busy
- Not scalable for large users

◆ 6. Disconnected Architecture

◆ Definition

- Data is fetched into memory
- Connection is closed immediately
- Operations happen on local data

◆ Flow

Open Connection → Fill DataSet → Close Connection → Work Offline

◆ **Example**

```
SqlConnection conn = new SqlConnection("connection_string");
```

```
SqlDataAdapter adapter = new SqlDataAdapter("SELECT * FROM Employees", conn);
```

```
DataSet ds = new DataSet();
```

```
adapter.Fill(ds); // connection opens & closes automatically
```

```
foreach(DataRow row in ds.Tables[0].Rows)
```

```
{
```

```
    Console.WriteLine(row["Name"]);
```

```
}
```

◆ **Advantages**

- Scalable
 - Reduces DB load
 - Suitable for web apps
-

◆ **Disadvantages**

- More memory usage
 - Data may become stale
-

◆ **7. Connected vs Disconnected (Important Interview Table)**

Feature	Connected	Disconnected
Connection	Always open	Open & close quickly
Performance	Faster	Slight overhead
Scalability	Low	High
Data Access	Real-time	Cached
Objects Used	DataReader	DataSet

Choosing a DataReader or a DataSet

When working with data, you often choose between **DataReader** and **DataSet**.

Feature	DataReader	DataSet
Connection type	Connected	Disconnected
Data access	Forward-only, read-only	Read/write, can navigate multiple tables
Performance	High (fast, low memory)	Moderate (more memory used)
Use case	Reading large volumes of data sequentially	Complex operations, multiple tables, relationships, offline work
Modification	Cannot update database directly	Can update database via DataAdapter

Rule of thumb:

- Use **DataReader** for **fast, one-way reading**.
- Use **DataSet** for **complex, offline operations** or **multiple related tables**.

◆ 8. When to Use What

✓ Use Connected (DataReader) When:

- Large data streaming

- Read-only operations
 - High performance needed
-

✅ **Use Disconnected (DataSet) When:**

- Working with multiple tables
 - Need offline processing
 - UI binding (GridView, etc.)
-

◆ **9. Real-World Insight**

In modern applications:

- ADO.NET is still used for:
 - High-performance systems
 - Stored procedure execution
 - Bulk operations
 - But often combined with:
 - **Entity Framework Core (ORM)**
 - **Dapper (Micro ORM)**
-

◆ **10. Summary**

- ADO.NET is the **foundation of data access in .NET**
 - Two key models:
 - **Connected → DataReader**
 - **Disconnected → DataSet**
 - Microsoft.Data.SqlClient provides SQL Server support
 - Choose architecture based on **performance vs scalability**
-

